САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа 5

Реализация межсервисного взаимодействия посредством очередей сообщений

Выполнил:
Люсин Дмитрий
Группа: К3339

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

В рамках домашнего задания необходимо подключить и настроить RabbitMQ или Kafka, а также реализовать межсервисное взаимодействие посредством очередей сообщений в микросервисном бэкенд-приложении.

Работа выполнялась для проекта приложения бронирования столиков в ресторанах. В качестве брокера сообщений был выбран RabbitMQ, так как он подходит для событийной коммуникации между микросервисами и удобен для локального запуска через Docker.

Ход работы

За основу была взята микросервисная версия приложения, реализованная в предыдущей лабораторной работе. В проект были добавлены RabbitMQ, библиотека amqplib для работы с очередями сообщений и отдельный Notification Service для обработки событий и создания уведомлений.

1. Состав сервисов

Сервис	Назначение	База данных
API Gateway	Единая точка входа для клиента	нет
Auth & User Service	Регистрация, вход, пользователи, JWT	auth_user.sqlite
Catalog Service	Рестораны, кухни, столики, рейтинг	catalog.sqlite
Menu Service	Категории меню и блюда	menu.sqlite
Media Service	Фотографии ресторанов	media.sqlite
Booking Service	Бронирования и события бронирований	booking.sqlite
Review Service	Отзывы и события отзывов	review.sqlite
Notification Service	Обработка событий и создание уведомлений	notification.sqlite

2. События RabbitMQ

Межсервисное взаимодействие через RabbitMQ реализовано в виде доменных событий. Сервис, в котором произошло важное действие, публикует событие в брокер сообщений. Другие сервисы подписываются на нужные события и выполняют свою логику независимо от исходного сервиса.

Событие	Кто публикует	Кто обрабатывает
booking.created	Booking Service	Notification Service
booking.cancelled	Booking Service	Notification Service
booking.status_changed	Booking Service	Notification Service
review.created	Review Service	Notification Service, Catalog Service
review.updated	Review Service	Notification Service, Catalog Service
review.deleted	Review Service	Notification Service, Catalog Service

3. Схема взаимодействия

REST-взаимодействие осталось для операций, где сервису нужен немедленный ответ, например при проверке пользователя, ресторана или столика. RabbitMQ используется для асинхронных действий: создания уведомлений и обновления рейтинга ресторана после изменений отзывов.



Рисунок 1 - Межсервисное взаимодействие через RabbitMQ

4. Пример сценария работы

При создании бронирования клиент отправляет запрос в API Gateway. Gateway перенаправляет запрос в Booking Service. После успешного создания записи Booking Service публикует событие booking.created в RabbitMQ. Notification Service получает это событие и сохраняет уведомление в своей базе данных.

При создании отзыва Review Service сохраняет отзыв в своей базе данных и публикует событие review.created. Catalog Service получает это событие и обновляет агрегированные поля ресторана: average_rating, reviews_count и rating_sum.

5. Проверка результата

Для проверки работы очередей можно выполнить вход пользователя, создать бронирование, затем открыть список уведомлений. После создания бронирования должно появиться уведомление, созданное Notification Service на основе события booking.created. Также можно создать отзыв и проверить, что после обработки события review.created у ресторана обновляются данные рейтинга.

```
POST /api/auth/login
POST /api/bookings
GET /api/notifications/my
POST /api/restaurants/{restaurantId}/reviews
```

```
GET /api/restaurants/{restaurantId}
```

Вывод

В ходе выполнения домашнего задания в микросервисное приложение был добавлен RabbitMQ как брокер сообщений. Было реализовано асинхронное взаимодействие между сервисами через события бронирований и отзывов.

Booking Service публикует события о создании, отмене и изменении статуса бронирования. Review Service публикует события об изменениях отзывов. Notification Service обрабатывает события и создаёт уведомления, а Catalog Service обновляет рейтинг ресторанов на основе событий отзывов. Таким образом, часть межсервисного взаимодействия была переведена с прямых REST-вызовов на событийную модель через очередь сообщений.